МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КАРПАТСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ВАСИЛЯ СТЕФАНИКА
ФАКУЛЬТЕТ МАТЕМАТИКИ ТА ІНФОРМАТИКИ

ЛАБОРАТОРНА РОБОТА 8–10

з дисципліни Професійна практика програмної інженерії

Реалізація бекенду застосунку для планування туристичних маршрутів із використанням геолокації

Виконав студент групи ІПЗ-42

Ницак Олег

Перевірив: Пікуляк Микола Васильович

Івано-Франківськ 2026

Мета роботи та реалізований функціонал

У лабораторних роботах 8–10 реалізовано Node.js сервер Trailway із REST API, PostgreSQL-базою, бізнес-логікою маршрутів і подорожей, автентифікацією, авторизацією, валідацією та обробленням помилок. Сервер інтегровано з React-клієнтом лабораторних 6–7, а структура основних даних продовжує модель лабораторної №5.

API приймає й повертає JSON. Публічні ендпоінти надають видимі туристичні локації та категорії. Приватні ендпоінти дозволяють користувачеві керувати власними маршрутами, створювати посилання, записувати подорожі й синхронізувати локальні зміни. Адміністратор керує каталогом. Перевірка ролі та власника виконується незалежно від доступності кнопок у клієнті.

Створення й оновлення маршруту охоплює план і впорядковані точки в одній транзакції. Контроль base_version запобігає втраті змін при роботі кількох пристроїв. Подорож отримує історичний знімок маршруту, тому видалення або зміна плану не переписує завершену історію. GPS-точки зберігаються окремо, мають час і сегмент та захищені від дублювання.

Автентифікація використовує короткочасний JWT access-token та ротацію випадкового refresh-token. Паролі проходять scrypt-хешування з випадковою сіллю. Поширені посилання також створюються криптографічно випадково, мають строк дії та можуть бути відкликані. У базі зберігаються лише хеші refresh-token і токенів посилань.

Стандартний запуск використовує вбудований PostgreSQL PGlite з постійним каталогом даних. Це дозволяє перевірити роботу SQL, ключів, JSONB і транзакцій без окремого сервісу. Підключення через DATABASE_URL використовує драйвер pg для окремого PostgreSQL. Під час виконання цієї роботи автоматичні та браузерні тести проведено на PGlite.

Архітектура серверної частини

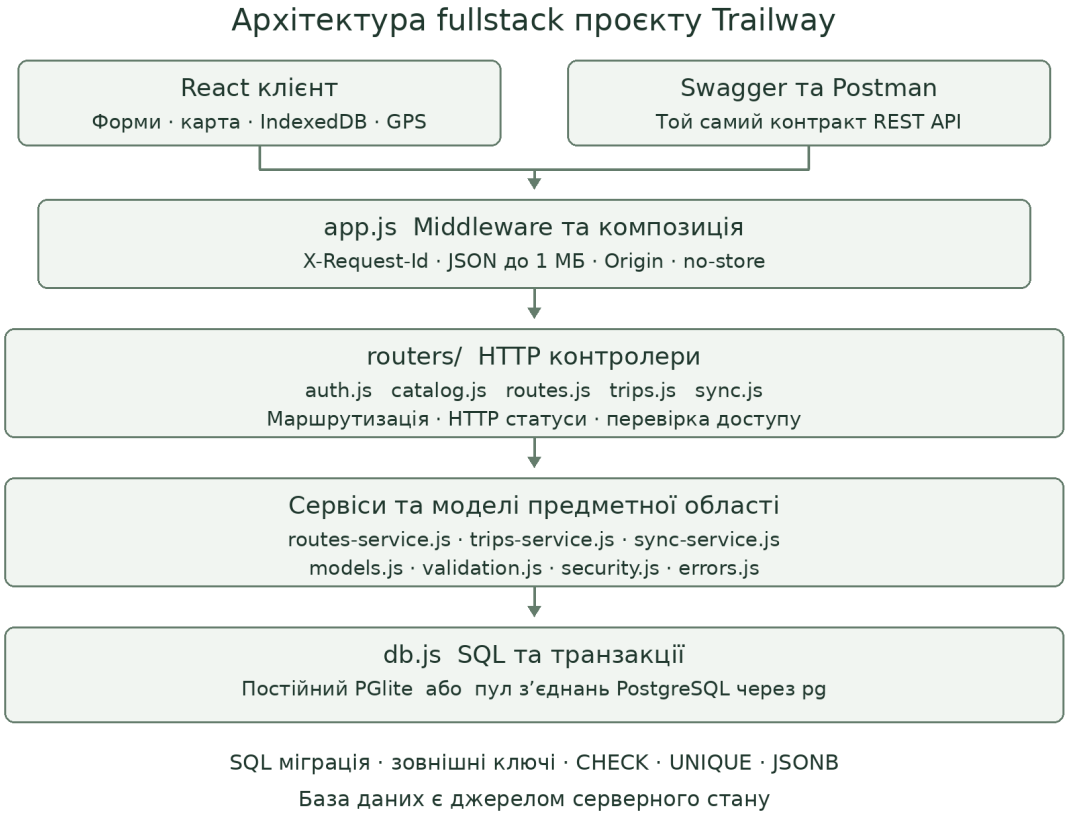


Рисунок 8.1 — Структура серверного застосунку й адаптери бази

app.js підключає спільні middleware та п’ять роутерів у server/src/routers. Вони приймають HTTP-запити й формують відповіді. routes-service.js, trips-service.js та sync-service.js реалізують бізнес-правила й транзакції. models.js описує дев’ять сутностей через JSDoc і константи станів; validation.js та security.js забезпечують перевірки даних і доступу.

db.js надає query, transaction та close для обох режимів бази. SQL-параметри передаються окремим масивом. Адаптер pg отримує окреме з’єднання пулу на транзакцію; PGlite виконує транзакційний callback з автоматичним commit або rollback. Фронтенд, Swagger і Postman використовують один REST-контракт.

Модель даних та міграція

Модель містить дев’ять основних сутностей попереднього етапу. Їх прикладні типи визначено в models.js, а фізичні таблиці, зв’язки, UUID, TIMESTAMPTZ, JSONB та ENUM реалізує SQL-міграція. refresh_tokens зберігає сесії, schema_migrations фіксує застосовану версію схеми. Повторний запуск не очищає записи.

Таблиця	Призначення та ключове правило
users	Email, scrypt-хеш, ім’я й роль туриста або адміністратора
place_categories	Унікальні коди й назви категорій
tourist_places	Категорія, координати й видимість; CHECK діапазонів
routes	Власник, стан, JSONB-геометрія, оцінки та version
route_points	Порядок 1–20; UNIQUE(route_id, position)
trips	Знімок маршруту, стан, час і фактичні показники
track_points	GPS; UNIQUE(trip_id, recorded_at)
share_links	Хеш випадкового токена, строк дії, відкликання
sync_operations	UNIQUE(user_id, client_operation_id) та збережений результат
refresh_tokens	Хеш токена сесії та ознака відкликання
schema_migrations	Номер застосованої міграції

Видалення Route каскадно вилучає точки й посилання, а в Trip встановлює route_id у NULL. Історичний route_snapshot залишається. Видалення TouristPlace лише обнуляє посилання RoutePoint на каталог, зберігаючи назву й координати точки. Для одного власника дозволено лише одну незавершену подорож, що контролюється частковим унікальним індексом.

Автентифікація та авторизація

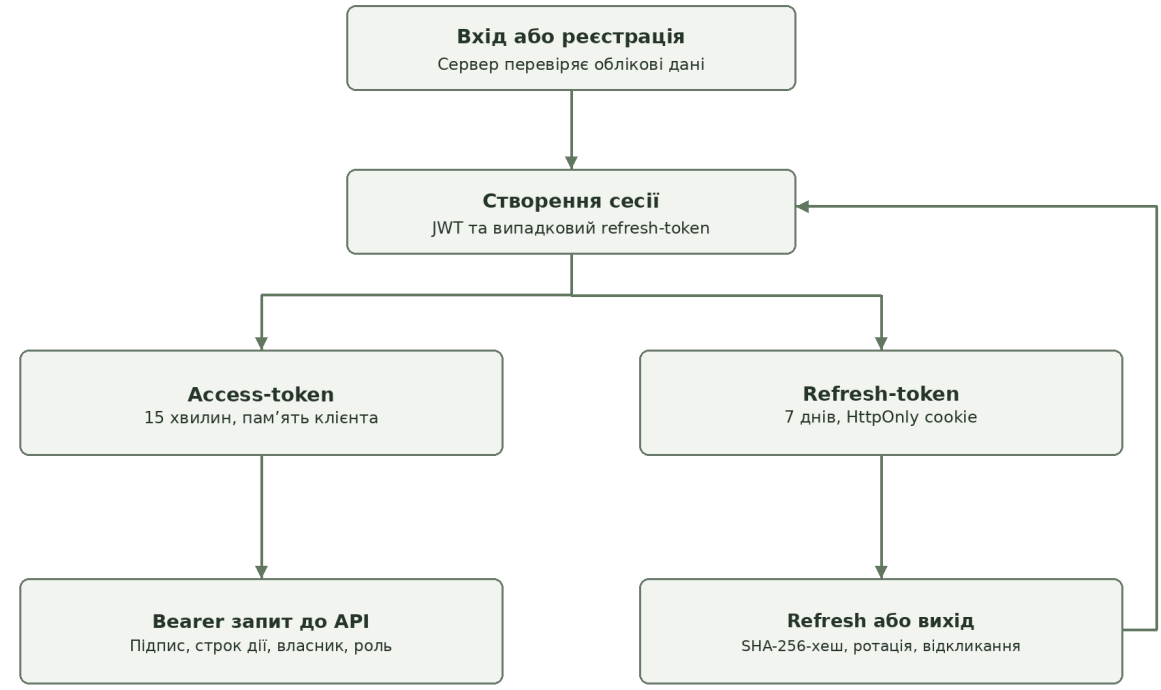


Рисунок 8.2 — Життєвий цикл облікової сесії

Після входу сервер повертає JWT access_token на п’ятнадцять хвилин і публічні дані користувача. Випадковий refresh-token на сім днів передається через HttpOnly cookie з SameSite=Strict і шляхом /api/auth. У production cookie має Secure. Access-token зберігається в пам’яті клієнта; у базі є лише хеш refresh-token.

Middleware перевіряє HS256-підпис, exp, тип токена, issuer і audience, після чого читає актуального користувача з бази. Операції маршруту додатково звіряють owner_id; адміністрування каталогу перевіряє роль admin. Самореєстрація не дозволяє призначити роль адміністратора.

Реалізація паролів та оновлення сесії

Паролі хешуються scrypt (N=16384, r=8, p=1) з випадковою сіллю. Ключі порівнюються через timingSafeEqual; відкриті паролі не потрапляють до бази або відповіді API.

```
export async function hashPassword(password) {
  const salt = randomBytes(16).toString('hex');
  const derived = await scrypt(password, salt, 64,
    {N: 16384, r: 8, p: 1});
  return `scrypt$16384$${salt}$${derived.toString('hex')}`;
}
export async function verifyPassword(password, encoded) {
  const [kind, cost, salt, hex] = encoded.split('$');
  if (kind !== 'scrypt' || cost !== '16384' || !salt || !hex)
    return false;
  const expected = Buffer.from(hex, 'hex');
  const actual = await scrypt(password, salt, 64,
    {N: 16384, r: 8, p: 1});
  return expected.length === actual.length &&
    timingSafeEqual(expected, actual);
}
```

Refresh виконується в транзакції з блокуванням запису токена. Повторне використання відкликаного або простроченого токена дає 401. Успіх відкликає попередній токен і видає нову сесію. Нижче наведено фрагмент контролера з routers/auth.js; роутер підключено з префіксом /api.

```
router.post('/auth/refresh', async (req, res) => {
  const refresh = getRefresh(req);
  if (!refresh)
    throw new ApiError(401, 'Увійдіть до облікового запису.');
  const tokens = await db.transaction(async tx => {
    const result = await tx.query(
      'SELECT * FROM refresh_tokens WHERE token_hash=$1 FOR UPDATE',
      [hashToken(refresh)]
    );
    const token = result.rows[0];
    if (!token || token.revoked_at ||
        new Date(token.expires_at) <= new Date())
      throw new ApiError(401, 'Сесія завершилась.');
    await tx.query(
      'UPDATE refresh_tokens SET revoked_at=now() WHERE id=$1',
      [token.id]
    );
    const resultUser = await tx.query(
      'SELECT * FROM users WHERE id=$1', [token.user_id]);
    return issueTokens(tx, resultUser.rows[0], secret);
  });
  sendSession(res, tokens);
});
```

Вихід відкликає refresh-token і очищає cookie; access-token діє до завершення свого короткого строку. Лімітер входу повертає 429 із Retry-After при перевищенні кількості спроб.

Бізнес-логіка маршрутів та транзакції

createRoute перевіряє стан і точки, створює Route, записує RoutePoint та повертає деталі. Оцінка довжини обчислюється за формулою гаверсинусів між сусідніми координатами. Геометрія має формат GeoJSON LineString. Після оновлення збільшується version та змінюється updated_at.

```
export async function updateRoute(tx, owner, id, body) {
  const existing = await routeDetail(tx, id, owner, {lock: true});
  integer(body.base_version, 'base_version', 1, 2147483647);
  if (body.base_version !== existing.version) {
    throw new ApiError(409,
      'Маршрут змінено на іншому пристрої. Оновіть дані.',
      {version: existing.version});
  }
  const r = routeInput(body), g = buildGeometry(r.points);
  await tx.query(`UPDATE routes SET
    name=$2, description=$3, state=$4, geometry=$5,
    distance_m=$6, duration_s=$7,
    version=version+1, updated_at=now()
    WHERE id=$1`,
    [id, r.name, r.description, r.state,
      JSON.stringify(g.geometry), g.distance_m, g.duration_s]
  );
  await replacePoints(tx, id, r.points);
  return routeDetail(tx, id, owner);
}
```

routeDetail перевіряє власника та за потреби блокує маршрут через FOR UPDATE. Оновлення допускається лише за рівності base_version і поточної version. При невідповідності повертається 409. Якщо точка посилається на відсутню або приховану локацію, транзакція відкочується разом із зміною самого маршруту.

PUT передає повний стан плану. PATCH поєднує надані поля з актуальним маршрутом та застосовує той самий контроль версії. DELETE повертає 204 і при повторному видаленні вже відсутнього маршруту, тому операція є ідемпотентною.

Контракт REST API

REST-інтерфейс групується за обліковими записами, каталогом, маршрутами, подорожами та синхронізацією. Детальний OpenAPI-контракт доступний за /api/openapi.json, а інтерактивний Swagger — за /docs. У запитах приватних операцій використовується Authorization: Bearer access_token.

Метод і шлях	Операція та доступ
POST /auth/register, /auth/login	Створення облікового запису та вхід
POST /auth/refresh, /auth/logout	Оновлення й відкликання сесії через cookie
GET /categories, /places	Публічний каталог; q, category, limit, offset
POST /places; PUT, DELETE /places/{id}	CRUD каталогу для адміністратора
GET, POST /routes	Список власника та створення маршруту
GET, PUT, PATCH, DELETE /routes/{id}	Деталі, оновлення і видалення власного плану
POST, GET /routes/{id}/shares	Створення та список посилань
DELETE /routes/{id}/shares/{shareId}	Відкликання власного посилання
GET /shares/{token}	Перегляд за чинним токеном без входу
GET, POST /trips; GET /trips/{id}	Історія, початок і деталі власної подорожі
POST /trips/{id}/pause, /resume, /finish	Контроль стану подорожі
POST /trips/{id}/track	GPS-пакет до 200 вимірювань
POST /sync	До 50 локальних операцій за один запит
GET /health	Стан API та режим бази

У таблиці наведено шляхи відносно префікса /api. Успішне читання повертає 200, створення — 201, видалення — 204. Помилки вхідних даних мають 400, відсутня сесія — 401, заборона доступу — 403, відсутній ресурс — 404, конфлікт — 409.

Подорожі та GPS-треки

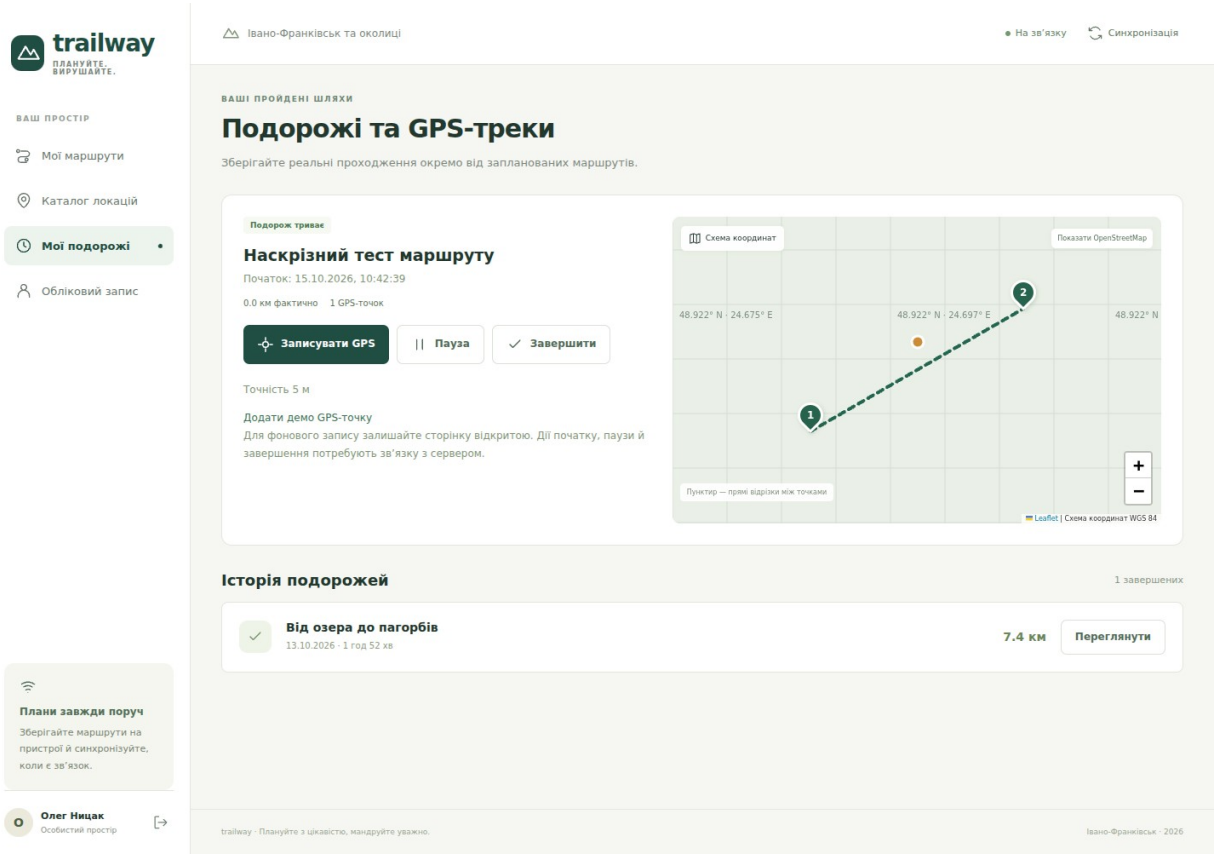


Рисунок 8.3 — Активна подорож у React-клієнті

Початок подорожі дозволено для готового маршруту свого власника. Сервер створює Trip зі знімком плану, часом початку й active_since. Пауза фіксує активну тривалість та переводить стан у paused; продовження створює новий відлік. Завершення встановлює finished_at, а повторне завершення повертає вже збережену подорож.

track_points має унікальність часу в межах подорожі. Повторна передача вимірювання не створює дубль. Сумарна відстань рахується лише між послідовними точками одного segment_id із точністю не гірше 100 м. Різні сегменти відокремлюють паузи чи розриви сигналу. Час до початку подорожі або некоректні координати відхиляються.

Наскрізний тест подав координату через браузерний механізм тестової геолокації, записав її в PostgreSQL і перевірив паузу, продовження та завершення. На фізичному телефоні необхідний дозвіл браузера і захищений контекст. Фонове виконання залежить від браузера; початок та переходи станів потребують мережі.

Синхронізація та ідемпотентність

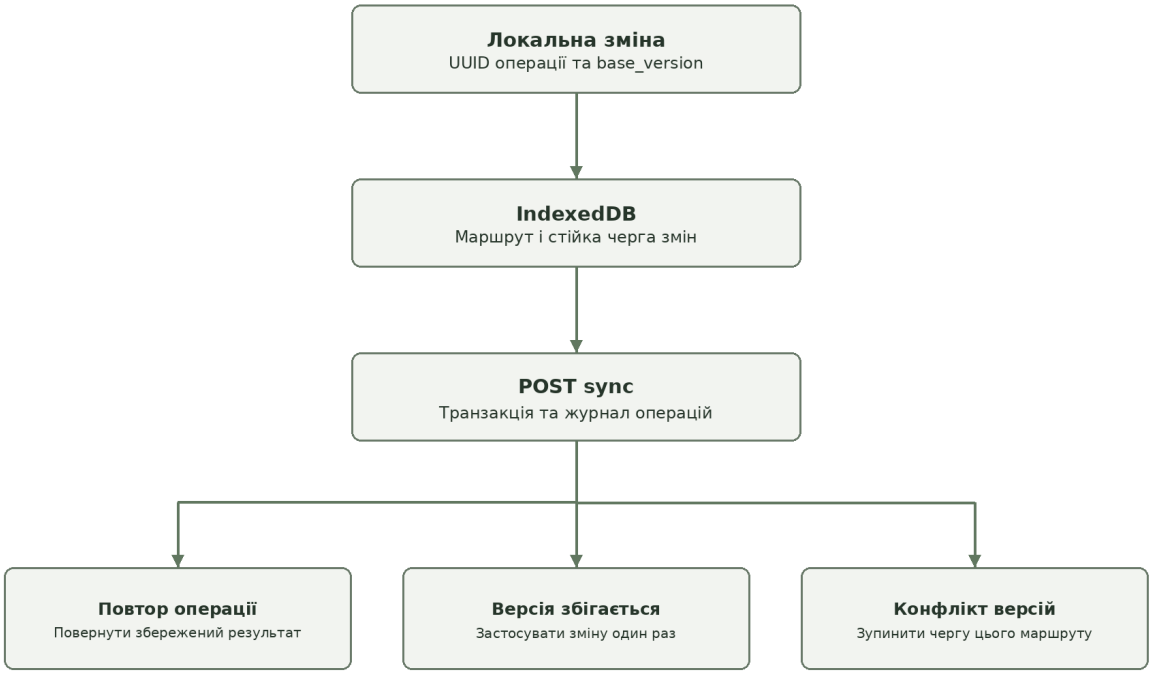


Рисунок 8.4 — Серверне оброблення повторів та конфліктів

Сервер блокує запис користувача та шукає пару user_id і client_operation_id у журналі. Контрольний хеш обчислюється зі впорядкованих JSON-ключів, тому їх перестановка не змінює зміст операції. Повтор запиту повертає збережений результат із replayed=true. Інші значення з тим самим ідентифікатором отримують 409. Конверти всього пакета перевіряються до застосування першої операції.

Для нової операції сервер використовує транзакцію та SAVEPOINT. Успішна зміна й журнал фіксуються разом. При очікуваній бізнес-помилці зміна відкочується до SAVEPOINT, а її статус conflict або failed зберігається. Це дозволяє повторити мережевий запит без подвійного застосування або втрати повідомлення про помилку.

Сервіс synchronizeRoutes підтримує create, update та delete. Очікувані помилки нормалізуються через publicError, тому результат failed або conflict не містить SQL і назв обмежень. Повтор запиту відтворює збережене повідомлення. Історія містить entity_type=route та UUID сутності без зовнішнього ключа до вже видаленого маршруту.

Валідація та оброблення помилок

Сервер перевіряє JSON-об’єкти, UUID, email, довжини полів, стани та кількість точок. Елемент points або operations не може бути null, масивом чи примітивом. Широта має діапазон від −90 до 90, довгота — від −180 до 180. limit і offset повинні бути цілими й допустимими. Додатково перевіряються власник, версія та поточний стан подорожі.

```
// Єдиний формат помилки клієнту
{
  status: 409,
  message: 'Маршрут змінено на іншому пристрої. Оновіть дані.',
  fields: { version: 2 },
  requestId: 'UUID запиту'
}
```

publicError узгоджує помилки HTTP та синхронізації. Некоректний JSON дає 400, тіло понад 1 МБ — 413, неправильний Content-Type — 415. SQL-помилки й внутрішній текст 500 приховано. Усі відповіді API мають Cache-Control: no-store. X-Request-Id зв’язує відповідь із журналом, який містить шаблон адреси, статус і тривалість без тіл запитів, паролів та токенів.

Перевірка	Очікувана реакція
Чужий маршрут	403 без читання або зміни даних
Прострочений чи змінений JWT	401
Стара версія Route	409 зі збереженням серверної копії
Широта поза діапазоном	400 без часткового запису
Відсутня локація точки	Rollback Route та RoutePoint
Повтор DELETE	204 без тіла відповіді
Повтор sync UUID	Збережений результат; без дубля
JSON, розмір та формат тіла	400, 413 або 415 з єдиним JSON-форматом
Друга незавершена Trip	409
Сторонній Origin для зміни	403

Результати автоматичних перевірок

15 жовтня 2026 року npm run verify успішно оновила експорт API, перевірила синтаксис, виконала 44 тести Node Test Runner і зібрала клієнт Vite 7.3.7. Набір охоплює схему, ролі, CRUD, транзакції, SQL-параметри, пагінацію, GPS, ротацію сесії, no-store, формати помилок та журналювання. Один тест виконує всі 19 запитів колекції Postman на реальному API з окремою базою.

```
tests 44
pass 44
fail 0
cancelled 0
skipped 0

Vite 7.3.7 production build completed
```

Браузерний сценарій завершив 16 перевірок, включно з мобільною навігацією, CRUD, GPS, офлайн-перезавантаженням і захистом від запізнілих відповідей. Swagger виконав GET /health. Помилок JavaScript не зафіксовано. Окремо npm run smoke:launch запустила справжній index.js, створила маршрут, перезапустила процес і прочитала той самий запис із диска. Робочі дані ці перевірки не змінюють.

Набір перевірок	Підтверджений результат
Серверний API	33 тести пройшли
HTTP-контракт та журнал	6 тестів пройшли
Клієнт та постійність бази	3 + 1 тести пройшли
Колекція Postman	1 тест виконав 19 запитів
Браузерні сценарії	16 перевірок пройшли
Справжній запуск і перезапуск	6 перевірок пройшли
Аудит npm-залежностей	0 відомих вразливостей

Інтерактивна документація API



Рисунок 8.5 — Swagger виконує запит перевірки стану API

Swagger відображає OpenAPI-контракт і дозволяє виконувати запити без написання окремого клієнта. На скриншоті розкрито GET /health: Try it out та Execute надсилають запит, а область відповіді показує HTTP-статус, JSON і заголовки. Усі операції документації звертаються до того самого API, що й React-клієнт.

OpenAPI описує вхідні дані, JSON-відповіді Session, PublicUser та Route і спільну схему помилки. PUT вимагає base_version, а координати мають явні межі. Для приватних запитів потрібно виконати POST /auth/login та вставити access_token в Authorize. Значення refresh-token не виводиться в JSON; cookie обробляє браузер.

Приклад відповіді показує, що сервер успішно виконав запит до бази даних. Заголовок no-store забороняє HTTP-кешування, а X-Request-Id дає змогу знайти відповідний запис журналу без передавання технічного тексту помилки користувачеві.

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store

{"status":"ok","database":"pglite"}
```

Адміністративний каталог та інтеграція

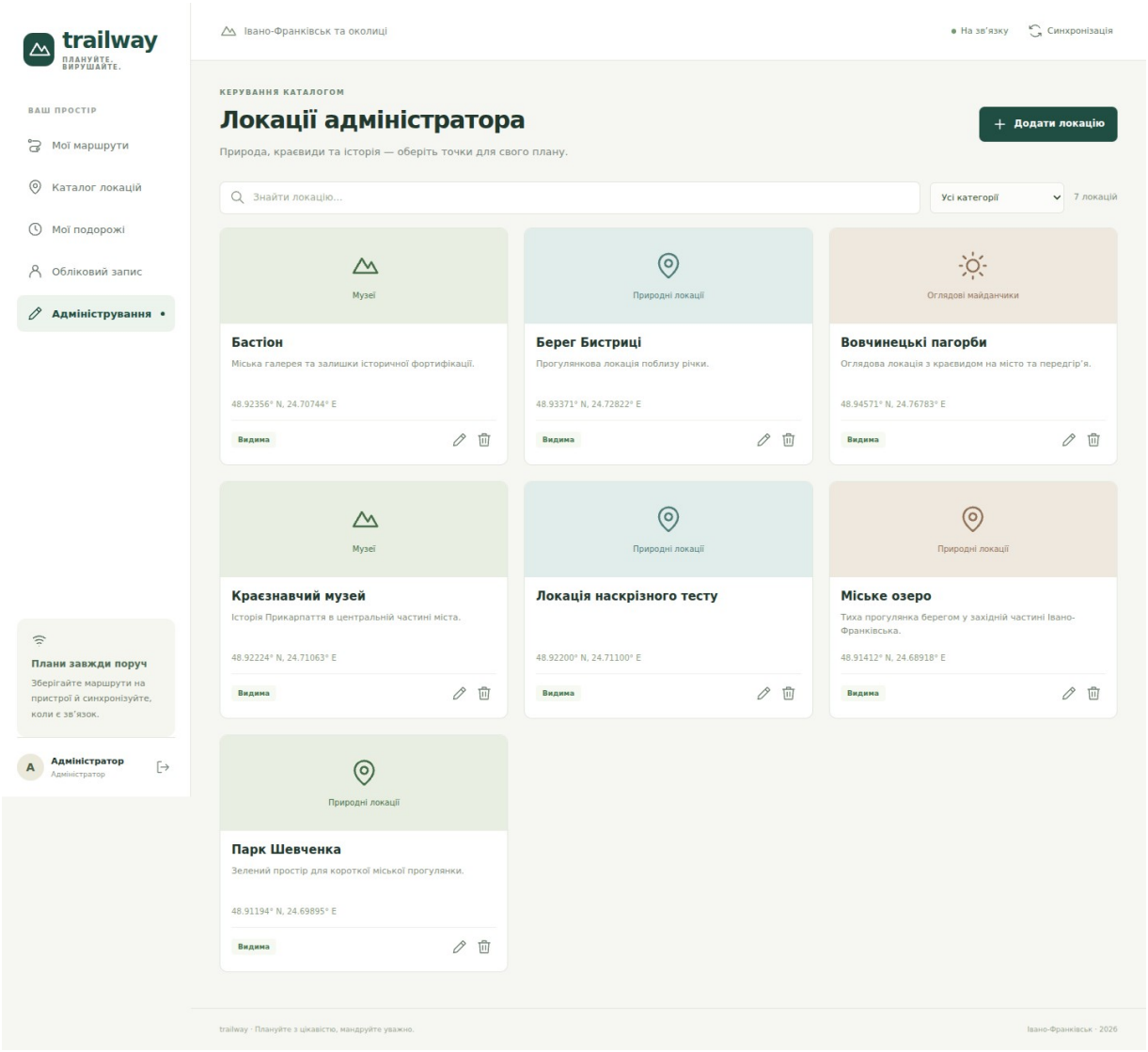


Рисунок 8.6 — Панель керування туристичними локаціями

Скриншот отримано після входу адміністратора й створення локації через UI. Сервер перевірив роль та записав дані до tourist_places. Адміністратор може змінити опис, категорію, координати й видимість або видалити запис. Публічний каталог показує лише is_visible=true.

React використовує той самий REST API, що описано в Swagger. Після запису інтерфейс повторно завантажує каталог та оновлює кеш IndexedDB. Спроба туриста викликати адміністративну операцію напряму завершується 403. Виправлення даних каталогу не переписує історичні знімки маршрутів або подорожей.

Структура повного проєкту

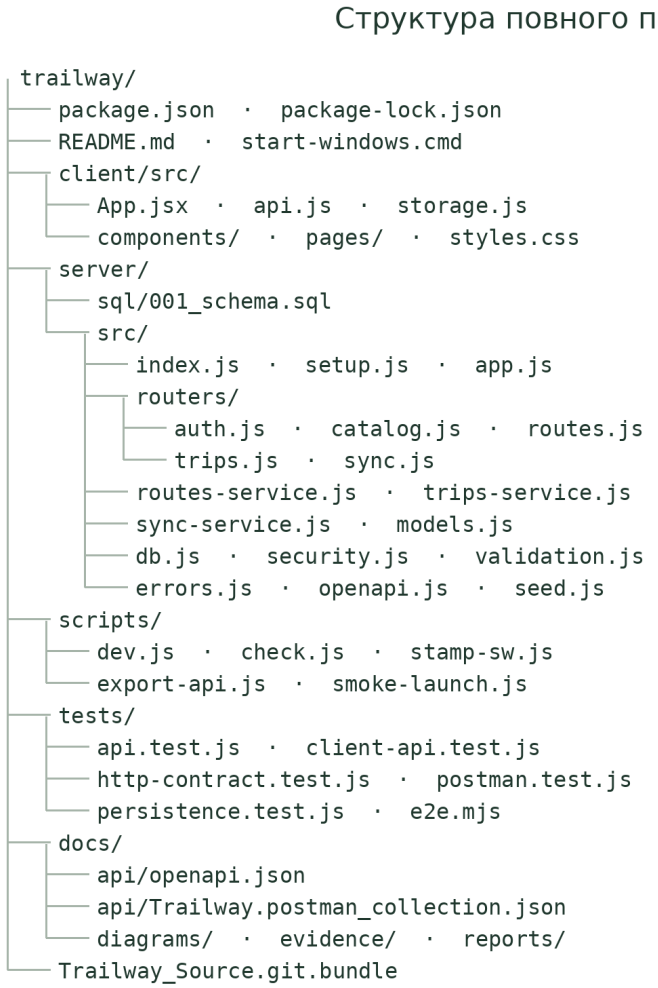


Рисунок 8.7 — Клієнт, сервер, документація й тести

HTTP-контролери зосереджено в routers, предметні операції — у сервісах. Каталог docs/api містить експорти контракту, docs/evidence — результати перевірки, docs/reports — звіти. Код запускається з кореневої папки trailway; залежності встановлюються за зафіксованим lockfile.

Запуск готового fullstack проєкту

```
npm ci
npm run setup
npm run build
npm start

# UI:      http://localhost:3001
# OpenAPI: http://localhost:3001/api/openapi.json
# Swagger: http://localhost:3001/docs
# Тести:   npm run verify
```

Параметр	Значення для перевірки
Інтерфейс та API	http://localhost:3001
Swagger	http://localhost:3001/docs
Турист	oleh@example.com / Tourist2026!
Адміністратор	admin@example.com / Admin2026!
Розроблення	npm run dev; клієнт 5173, API 3001
Доказ запуску	npm run smoke:launch

Потрібен Node.js 22.13 або новіший. На Windows можна запустити start-windows.cmd із папки trailway. PGlite зберігає дані в data/pgdata, тому перезапуск не втрачає маршрути. Для окремого PostgreSQL задається DATABASE_URL у .env; зовнішній сервер у цій перевірці не запускався. Основний готовий сценарій перевірено на PGlite без Docker.

Відтворення запитів у Postman

Файл docs/api/Trailway.postman_collection.json імпортується через Import. Після запуску сервера base_url дорівнює http://localhost:3001/api. Запити виконуються послідовно або через Collection Runner. Тести колекції перевіряють HTTP-статус, а скрипти автоматично передають токен, UUID та версію між операціями.

Змінна колекції	Звідки береться значення
access_token	Вхід та оновлення сесії
route_id, route_version	Створення та редагування маршруту
share_id, share_token	Створення посилання
trip_id	Початок подорожі

Початок сценарію має такий HTTP-зміст. У колекції значення Bearer підставляється автоматично; refresh-token зберігається в cookie jar і не копіюється вручну.

```
POST /api/auth/login
Content-Type: application/json

{"email":"oleh@example.com","password":"Tourist2026!"}

GET /api/routes?limit=10&offset=0
Authorization: Bearer <access_token>
```

Усі 19 запитів колекції виконано тестом проти справжнього Express API: від входу й CRUD до подорожі, відкликання посилання, повторного DELETE, refresh та logout. Виконання через Postman у робочій базі залишає завершену тестову подорож; створений маршрут видаляється наприкінці. npm run export:api відтворює обидва JSON-файли документації.

Репозиторій та висновки

Архів містить клієнт, сервер, SQL-міграцію, package-lock.json, тести, README, OpenAPI, колекцію Postman та звіти. Trailway_Source.git.bundle є повним локальним Git-репозиторієм з історією та фінальними вихідними файлами. Його можна відтворити командою git clone без зовнішніх інтеграцій. Віддалений URL не створено; README містить команди для самостійного додавання власного remote. Для запуску застосунку віддалений репозиторій не потрібен.

```
git clone Trailway_Source.git.bundle trailway-repository
cd trailway-repository
npm ci
npm run build
npm start
```

У лабораторних роботах 8–10 реалізовано модульний бекенд та завершено інтеграцію з фронтендом 6–7. Сервер керує обліковими сесіями, ролями, власниками, маршрутами, каталогом, GPS-подорожами й синхронізацією. Транзакції та версії захищають цілісність даних. Тести підтвердили CRUD через HTTP і браузер та збереження записів після повного перезапуску сервера.

Реалізація зберігає послідовність попередніх лабораторних і дає змогу відтворити результат на комп’ютері без Docker. Карта координат і оцінки плану не враховують доступність проходу та рельєф; запис GPS залежить від дозволів і відкритої сторінки браузера. Ці межі відображено в інтерфейсі та інструкції запуску.

Використані матеріали

Умова Lab 8-10 (1).docx; звіт лабораторних 6–7 як зразок оформлення; Lab5.pdf як модель даних попереднього етапу.

Express middleware та помилки: https://expressjs.com/en/5x/guide/using-middleware/ ; https://expressjs.com/en/5x/guide/error-handling/

PostgreSQL схему реалізовано в server/sql/001_schema.sql; відповідності сутностей наведено у models.js.

PGlite: https://pglite.dev/docs/api

Node.js crypto: https://nodejs.org/api/crypto.html